

Caelum-Agent 阶段性开发汇报

项目	Caelum-Agent —— Windows 桌面操作智能体 (CLI)
汇报周期	2026-07-09 ~ 2026-07-11 (v8 重写落地周)
报告日期	2026-07-12
代码基线	main @ cd867d3

一、概述

本周交付的是**重写后的 Caelum-Agent**。上一阶段的技术预研已验证三条关键结论（见第二节），据此推倒旧实现，本周按 v8 设计整体重写：三天内完成核心实现与两轮加固，系统已可端到端运行：

自然语言指令 → Kimi K2.6 规划 → Playwright / Windows / Filesystem 三 MCP 执行 → GUI-Actor-3B 视觉定位 → 验证收尾 → 答案输出

首周关键数字：

指标	值
Git 提交	98 (7/9: 3 • 7/10: 50 • 7/11: 45)
Python 代码	78 文件 / 15,615 行 (自研 6,602 + vendored GUI-Actor 2,008 + 测试 7,005)
测试	30 文件 / 359 passed (含 8 个集成冒烟测试)，行覆盖率 90%，全套约 2 分钟
覆盖模块	agent / eventbus / mcp_client / ui_detector / main 全链路

二、前阶段验证结论与重写决策

本周的重写并非平地起楼，而是基于上一阶段技术预研验证的三条关键结论：

验证结论	对应架构决策
Kimi 工具调用必须走官方格式 ：用通用 function-calling 格式 + prompt 约束的方式失败率高	接入 Kimi Formula 官方工具 (web-search / fetch / memory / rethink / code-runner 等)；本地工具 (CompleteTask、DesktopInteract、RequestHumanHelp、CodeRunner) 统一按官方 function calling 格式注册
OmniParser 元素检测效果不及 GUI-Actor	放弃 OmniParser，视觉定位统一为 GUI-Actor-3B (+ 三态 Verifier)，以 "SoM 标注截图 + DesktopInteract" 形成检测—执行闭环
在通用核心循环上二次开发不如重写定制 ReAct 循环	不复用通用 agent 框架，自研 Perceive → Reflect → Think → Act → Verify 五段循环 + 8 态 FSM；验证、反思、人工交接等环节均按本项目语义定制

这三条结论直接决定了 v8 的技术选型，也是本周“重写而非续写”的原因——旧实现中通用的工具调用层、OmniParser 感知链路和框架式循环被整体替换。

新旧架构逐层对照

旧设计 ([docs/old/system-design.md](#)，第一阶段第 1 周，基于 Agent-Controller SDK + smolagents 框架) 与 v8 的逐层对照：

层	旧架构	v8（当前实现）
Agent 内核	smolagents CodeAgent + LiteLLMModel （通用 function-calling + prompt 约束）	自研五段 ReAct 循环 + 8 态 FSM， 官方工具格式
LLM	DeepSeek 文本模型为主，视觉模型仅 fallback	Kimi K2.6 单一多模态大脑
规划	Manager/Worker 分层规划 + BestOfN 采样评审	单层 ReAct，仅失败时触发 Reflect
浏览器	playwright skill	Playwright MCP，a11y 快照优先
桌面控制	pyautogui / pyperclip + 自研 UIA 调用	Windows-MCP（Snapshot→label 制）
感知	OpenCV/OmniParser 检测 + PaddleOCR/EasyOCR + UIA 注入 + 二维码 + 容器轮廓，五源 IoU 融合	GUI-Actor-3B + RapidOCR + a11y SoM 标注图直送多模态 LLM
Grounding	7 专家 Mixture-of-Grounding 路由 （Explicit/Text/UIA/VisionDetector/PlaywrightDom/VisionLLM/UI-TARS）	a11y ref / Snapshot label 双结构通 + GUI-Actor • SoM 统一视觉通道
动作验证	前后截图 diff（changed/unchanged）	GUI-Actor Verifier 三态判定
记忆	JSON 文件 + 自研反思/聚类提技能	Kimi memory 工具 + SQLite 兜底 + ChromaDB；AutoSkill 生成 SKILL
MCP 生态	6 个 server （playwright/uia/context7/duckduckgo/firecrawl/open_websearch）	3 个 server （playwright/windows/filesystem 搜索/抓取交给 Kimi Formula 内置
配置	.env 开关海	config.yaml + Pydantic 校验
安全/人机	文档中基本缺位	四级风险分级 + 急停（Ctrl+C 取消 进程）+ RequestHumanHelp 人工 接

被砍掉的过度工程：7 专家 grounding 路由、Manager/Worker 分层规划、五源感知融合 + IoU 去重、6 个 MCP server——都是"检测器不可靠 / 规划不稳"时代的补丁层。Kimi K2.6 + GUI-Actor-3B 两个更强的单点模型，让这些路由、融合、分层整体失去存在必要，复杂度大幅下降。

保留下来的设计基因（旧文档延续至 v8，方向判断被验证正确）：文本优先（a11y/UIA 先行、视觉兜底）、确定性元素引用（**btn_1** → a11y ref / Snapshot label / SoM 标号）、记忆驱动进化（trajectory 聚类提技能 → AutoSkill 生成 SKILL.md）、降级哲学（任何子系统失败不阻塞主任务）、动作后验证（截图 diff → Verifier 三态）。

三、v8 架构落地情况

设计文档 **docs/designs/desktop_agent_v8.agent.final.md** 的全部核心层均已实现并验证：

层	选型	状态
LLM 大脑	Kimi K2.6 API（Formula 内置工具 + 本地 Function 工具）	

层	选型	状态
浏览器控制	Playwright MCP（a11y 快照优先）	✓
桌面控制	Windows-MCP（Snapshot→label 强制）	✓
文件系统	server-filesystem MCP	✓
UI 检测	GUI-Actor-3B + 三态 Verifier（Transformers 原生推理）	✓（RTX 4090 / torch cu130 已验证）
OCR	RapidOCR（ONNXRuntime CPU）	✓
截图	mss + Pillow 压缩	✓
记忆	Kimi memory 工具 + SQLite 兜底	✓
反思	Kimi rethink 工具 + 本地记录	✓
状态机	8 态 FSM（含 WAITING_HUMAN）	✓
事件总线	asyncio EventBus（优先级队列 + 中间件链）	✓
MCP 多路复用	单事件循环 3 stdio 连接 + 健康监控自动重连	✓
急停	pynput 全局监听 + 任务级取消（Ctrl+C 不杀进程）	✓

四、每日进展

7/09 • 重写落地（3 提交）

- 仓库初始化与 v8 核心重写落地：`agent/`（orchestrator、状态机、感知、安全、急停、工具、记忆、反思、技能）、`eventbus/`、`mcp_client/`、`ui_detector/` 按新架构整体重写就位。
- audit、安全分级、技能配置首轮打磨。

7/10 • 核心贯通 + 第一轮加固（50 提交）

- 运行稳定性**：KillSwitch 全面 asyncio 串行化（防抖竞态、线程安全、Ctrl+C 改为取消当前任务而非杀进程）；EventBus 优先级队列 + 中间件；MCP 后台健康监控与断线重连、取消/重连竞态修复。
- 安全**：RestrictedCodeRunner 运行时导入白名单 + 两轮沙箱逃逸封堵；API key 从 repr 脱敏；工具风险分级分类器；MCP 调用前 kill-switch 前置检查。
- 视觉感知**：Perception 生成 SoM 标注截图并送入 LLM（标注缺失回退原图）；VerifierVerdict 三态判定贯通 detector→perception→orchestrator；GUI-Actor 惰性加载（纯计算任务不再常驻 7.6GB 显存）；`topk_points` 扁平坐标解析修复。
- 桌面控制**：DesktopInteract 工具（SoM 标号 → 坐标动作）；强制 Snapshot→label 并拦截"幻觉成功"；windows-mcp box-drawing 快照解析修复（此前 UI 树近乎为空）；上游 `tree_node` 崩溃的 stderr 噪声过滤；CJK 窗口名解析保护。
- 记忆与反思**：Kimi memory / rethink Formula 工具接入，SQLite 本地兜底。
- CLI 基础**：stdout/stderr 强制 UTF-8（Unicode 输出不再崩）；`--yes / --yes-destructive` 非交互批准。
- 测试基建**：共享 fakes + conftest fixtures，测试文件批量迁移；Kimi API / MCP / 生命周期的集成冒烟测试。
- 日志治理**：全仓库 6 处静默吞异常改为日志记录。
- 文档**：完整 README 上线。

7/11 • 交互体验升级 + 人工交接（45 提交）

1. **CLI 界面升级 (rich, spec → plan → 实现全流程)**: 事件驱动 CLIPresenter (banner、工具调用、确认、答案面板); 动态内容 markup 注入转义; 修复 rich Live 与 `input()` 冲突导致的确认卡死 (确认时挂起 spinner); 启动 MCP 连接状态行; 启动失败中途清理。
2. **收尾机制改革**: 模型自行决定何时跳过 Verify (CompleteTask 工具) —— "你好"类寒暄不再空跑验证; 提示词强化 Playwright-first; 针对模型把工具调用"写成纯文本"的兜底正则解析 + 防鹦鹉学舌措辞。
3. **RequestHumanHelp 人工交接 (本阶段重点交付)**: 完整 spec → plan → 三任务 TDD, 每个任务经 spec 合规 + 代码质量双评审。详见第五节。
4. **执行环境感知**: 模型通过系统提示词知道自己是否处于交互终端——管道/脚本调用时禁止使用 RequestHumanHelp, 直接输出"需用户手动完成"的收尾说明, 避免空跑一轮误导性的 `[cancelled]`。
5. **工作区治理**: agent 草稿文件 (ally 快照、抓取内容等) 统一写入 `data/cache/` (提示词约束 + CodeRunner 子进程 cwd 结构兜底), 项目根目录不再残留。
6. **性能与体验**: transformers `use_fast` 图像处理器 + 慢速兜底; `ui_detector.preload` 热加载 (首次 SoM 点击不卡顿); 技能学习后台化; 重复工具调用批短路。
7. **易用性**: `--yes-destructive` 更名为 `--yes-all`。
8. **测试覆盖**: 接入 pytest-cov; `logging_config` 26%→100%; `perception` 60%→77%; `llm_client / detector / mcp_client / main` 覆盖补齐。
9. **文档**: README 中英双语。

五、重点交付: RequestHumanHelp 人工交接工具

背景: agent 操作过程中必然遇到必须人类介入的步骤 (登录、扫码、验证码、2FA、系统授权弹窗)。此前只能靠失败阈值被动转入 WAITING_HUMAN, 体验差且无法恢复。

方案 (spec: docs/superpowers/specs/2026-07-11-request-human-help-design.md):

- **工具调用即暂停**: 模型调用 `RequestHumanHelp(question, options)` → orchestrator 校验参数 (2-4 选项) → 状态机转 `WAITING_HUMAN` → 同步调用 CLI 注入的 callback (与 `confirm_interactive` 同线程模型) → `finally` 恢复 `EXECUTING` → 回答以 `Human answered: ...` 回灌 history, 任务带完整上下文继续。
- **msvcrt 原生菜单**: 标题 `需要你的帮助: {question}`, ↑↓ 选择, 最后一行恒为 `type something:` 可直接打字 (含中文输入法)、退格删字、空输入回车无效、ESC/Ctrl+C 取消。按键经可注入 `getch` 读取, 全量单元测试覆盖。
- **四态工具返回**: `Human answered: [cancelled]` (ESC/非 TTY) / `[unavailable]` (无 callback) / `[error]` (参数非法), 模型可据此分支处理。
- **降级路径**: 非 TTY 不弹菜单直接返回; 无 presenter 时退化为编号 `input()`; callback 抛异常降级为 `[cancelled]` 且状态一定恢复。
- **提示词契约**: 模型被告知"永远不要自己提供自由文本选项" (CLI 自动追加), 并根据回答决定继续或收尾。

实现质量: 3 个实现任务 × (spec 合规评审 + 代码质量评审) + 最终整体评审 (VERDICT: READY); 新增 18 个测试, 全套 359 passed。

六、开发方法备注

- 所有功能遵循 **spec → plan → TDD → 双评审** 流程 (superpowers 工作流); 实现任务由独立 subagent 执行, 主会话只做编排与验收。
- 高频提交、小步快跑: 98 个提交全部带语义化 message (feat/fix/test/chore/docs)。
- 安全红线保持完整: `config.yaml` 永不入库; destructive 动作强制逐字确认; `--yes` 不涵盖 destructive (需显式 `--yes-all`); PowerShell/Registry 等高危 MCP 工具默认建议排除。

七、遗留事项与下阶段计划

#	事项	状态/ 优先级
1	三阶段验收任务端到端跑通 ：旧设计定下的终局目标——知乎（登录→发文→配图→搜索互动）、WPS（新建→编辑→样式→导出 PDF）、微信（搜索服务号→关注→私信），以 v8 引擎跑通，并建立四维指标基线（速度 / Token / 稳定性 / 扩展性）	下阶段 · 最高
2	真实终端手动验收 ：RequestHumanHelp 菜单（↑ ↓ /打字/ESC/Ctrl+C）+ 草稿文件落 <code>data/cache/</code> 两点需人工跑一次（自动化无法覆盖真键盘）	待执行 · 高
3	windows-mcp 上游 <code>tree_node</code> bug 的 issue 提交（草稿： <code>docs/windows_mcp/upstream-tree-node-issue.md</code> ，本地噪声过滤已上线）	待执行 · 中
4	可选加固：filesystem MCP 允许目录收紧（移除项目根，只留 <code>data/cache</code> + Documents）	待定 · 低
5	被动 WAITING_HUMAN（连续失败阈值）接入同一菜单交互	下阶段
6	人问问答持久化（接入 <code>audit_log</code> / 记忆）	下阶段
7	<code>data/cache/</code> 清理策略（保留期 / 容量上限）	下阶段
8	噪声过滤之外的 windows-mcp 树缺失根治（跟随上游修复）	跟踪上游

汇报人：开发组（Claude Code 协作） · 数据截至 2026-07-12（commit `cd867d3`；全套 359 passed 已复验）